



4. Remember-me 자동 로그인·비밀번호 재설정

담당: 이정하

개요

"로그인 상태 유지"(Remember-me) 자동 로그인과 비밀번호 재설정을 함께 다룬다. 둘은 별개 기능이 아니라 `password_updated_at` 하나로 맞물려 돌아간다 — 비밀번호를 재설정하면 자동 로그인 토큰 버전이 바뀌어 전 기기의 쿠키가 한번에 무효화된다.

Remember-me는 체크 시 30일짜리 자동 로그인 쿠키를 발급하고, 브라우저 재방문 시 세션을 복원한다. **토큰 테이블 없이** HMAC 서명만으로 위조를 막는 자체 쿠키 스킴이다.

- 구현 파일: `security/RememberMeCookie.java`, `security/RememberMeInterceptor.java`,
`auth/service/AuthService.java`, `auth/service/PasswordResetService.java`,
`common/validation/PasswordPolicy.java`

핵심 포인트

- 쿠키 값은 `userId.version.HMAC-SHA256(userId:version)` 세 조각 — 서명이 맞지 않으면 위조로 간주
- version은 `password_updated_at` 의 epoch 초 → 비밀번호를 변경하면 값이 바뀌어 이전에 발급된 모든 기기의 쿠키가 자동 폐기
- 복원 시에도 계정이 `ACTIVE` 인지, 쿠키 version이 현재 계정 version과 같은지 이중 확인
- HMAC 비밀키는 기본값 없이 `@Value` 로 강제 주입해 미설정 시 앱이 기동하지 않는다 (fail-fast)
- 쿠키는 `HttpOnly` 로 발급해 JS에서 접근 불가

주요 코드

| 위치: `security/RememberMeCookie.java` 28~37행 (`issue`) + 42~55행 (`resolve`)

```

// 발급: 발급 시점의 계정 버전을 토큰에 심는다
public void issue(HttpServletRequest response, Long userId, long version) {
    String token = userId + "." + version + "." + sign(userId + ":" + version);
    Cookie cookie = new Cookie(COOKIE_NAME, token);
    cookie.setHttpOnly(true);
    cookie.setPath("/");
    cookie.setMaxAge(MAX_AGE_SECONDS); // 30일
    response.addCookie(cookie);
}

// 검증: 서명 불일치 = 위조
public Token resolve(HttpServletRequest request) {
    for (Cookie c : request.getCookies()) {
        if (!COOKIE_NAME.equals(c.getName())) continue;
        String[] parts = c.getValue().split("\\.");
        if (parts.length != 3) return null;
        if (!sign(parts[0] + ":" + parts[1]).equals(parts[2])) return null;
        return new Token(Long.parseLong(parts[0]), Long.parseLong(parts[1]));
    }
    return null;
}

```

위치: `auth/service/AuthService.java` 96~101행 (`restoreSession`) + 103~106행 (`currentTokenVersion`)

```

// 복원: 상태·버전이 모두 맞을 때만 세션 재생성 (비번 변경 후 옛 쿠키 거부)
public void restoreSession(HttpSession session, Long userId, long tokenVersion) {
    userMapper.findById(userId)
        .filter(user -> STATUS_ACTIVE.equals(user.getStatus()))
        .filter(user -> tokenVersion == currentTokenVersion)

```

```

    sion(userId))
        .ifPresent(user -> sessionService.saveUser(session, toSessionUser(user)));
}

public long currentTokenVersion(Long userId) {
    return userCredentialMapper.findByUserId(userId)
        .map(c -> c.getPasswordUpdatedAt() == null
            ? 0L : c.getPasswordUpdatedAt().toEpochSecond(ZoneOffset.UTC))
        .orElse(0L);
}

```

비밀번호 재설정

이메일로 재설정 링크를 보내 비밀번호를 바꾸는 기능. remember-me와 같은 철학으로 토큰 원문을 DB에 남기지 않고, 일회용·30분 만료·재전송 쿨다운까지 갖춘 구조다.

- 토큰은 256bit `SecureRandom` → URL-safe Base64로 발급하고, **DB에는 SHA-256 해시만 저장** (원문은 메일 링크에만 존재 — DB 유출 시에도 링크 복원 불가)
- `used_at IS NULL` 조건이 걸린 **조건부 UPDATE로 일회용을 보장** — 동시에 두 번 제출해도 정확히 한 번만 성공
- 60초 재전송 쿨다운으로 메일 폭탄 방지, 정지·탈퇴 계정에도 "전송됨"과 동일한 응답을 줘 계정 상태 비노출
- 새 비밀번호는 공통 `PasswordPolicy` 로 닉네임·이메일 아이디 포함 여부까지 검사
- 저장 시 `password_updated_at` 갱신 → **위 remember-me 토큰 버전이 바뀌어 재설정 성공과 동시에 기존 자동 로그인 쿠키가 전 기기에서 무효화** — 두 기능이 맞물리는 지점

위치: `auth/service/PasswordResetService.java` 91~108행 (`issueToken`)

```

// 발급: 쿨다운 → 기존 토큰 무효화 → 해시만 저장, 원문은 메일 링크로
private String issueToken(Long userId) {
    PasswordResetToken latest = passwordResetTokenMapper.findLatestActiveByUserId(userId).orElse(null);
    if (latest != null && latest.getCreatedAt()
        .plusSeconds(RESEND_COOLDOWN_SECONDS).isAfter(L

```

```

ocalDateTime.now())) {
    return null; // 60초 내 재요청 → 메일 폭탄 방지
}
passwordResetTokenMapper.invalidateActiveByUserId(userId); // 유효 링크는 항상 최신 1개만

byte[] bytes = new byte[TOKEN_BYTES]; // 256bit
secureRandom.nextBytes(bytes);
String token = Base64.getUrlEncoder().withoutPadding().
    encodeToString(bytes);

PasswordResetToken row = new PasswordResetToken();
row.setUserId(userId);
row.setTokenHash(sha256Hex(token)); // DB에는 해시만
row.setExpiresAt(LocalDate.now().plusMinutes(tokenTtlMinutes)); // 30분
passwordResetTokenMapper.insert(row);
return token;
}

```

위치: `auth/service/PasswordResetService.java` 116~136행 (`resetPassword`, `credential/닉네임 조회부 축약`)

```

// 사용: 조건부 UPDATE 한 방으로 일회용 보장 (동시 제출 경쟁 안전)
if (passwordResetTokenMapper.markUsed(row.getTokenId()) != 1) {
    throw new BusinessException(ErrorCode.AUTH_RESET_TOKEN_USED);
}

if (PasswordPolicy.containsProfileInfo(newPassword, nickname, credential.getLoginEmail())) {
    throw new BusinessException(ErrorCode.AUTH_PASSWORD_TOO_GUESSABLE);
}

credential.setPasswordHash(passwordEncoder.encode(newPassword));

```

```
// update()가 password_updated_at = NOW()를 함께 갱신
// → remember-me 토큰 버전이 달라져 기존 자동 로그인 쿠키가 전부
무효화된다
userCredentialMapper.update(credential);
```

시연 화면

1. `/login` 의 "로그인 상태 유지" 체크박스

비밀번호

공용 PC가 아닌 개인 PC에서만 사용해 주세요.

☒ 로그인 상태 유지

로그인

2. 개발자도구(F12) Application → Cookies → REMEMBER_ME 값이 7.1753142400.f3ab... 처럼 점 3조각(userId.version.서명)으로 저장됨

Application

Manifest

Service workers

Storage

WebMCP

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

http://localhost:8080

Private state tokens

Interest groups

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking mitig...

Notifications

Payment handler

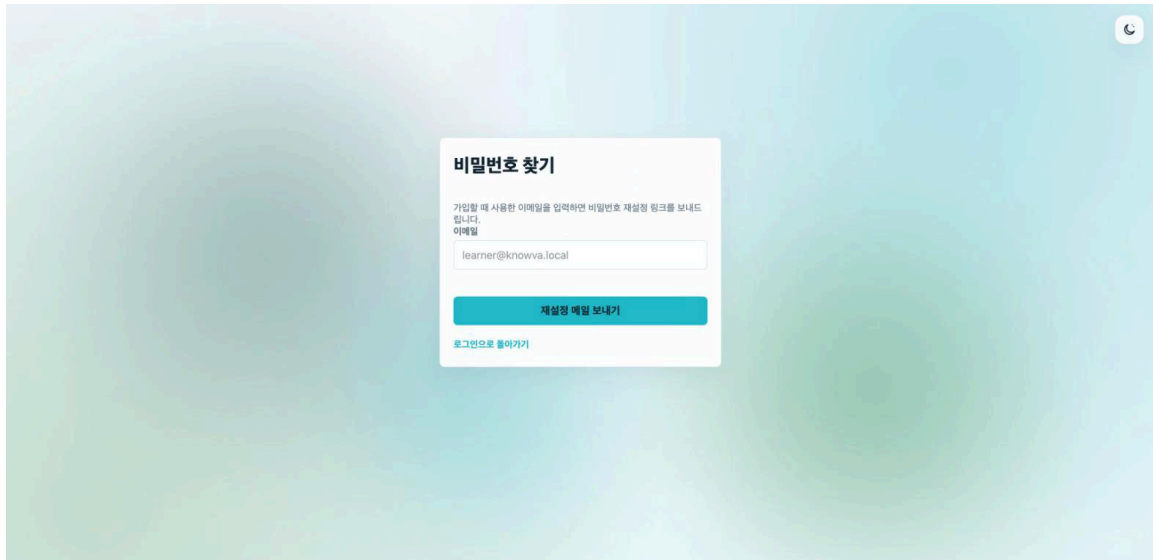
Periodic background s...

Filter

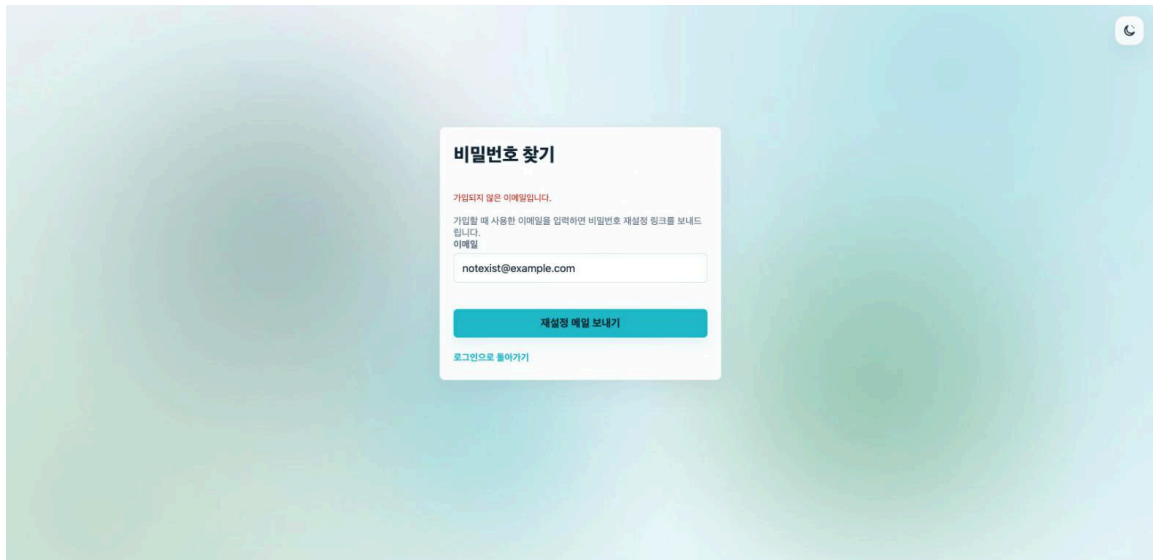
Only show cookies with an issue

Name	Value	Dom...	Path	Expir...	Size	Http...	Secure	Sam...	Partit...	Cros...	Priority
age	25	local...	/	2027...	5						Medi...
ajs_anonymous_id	9ba1cbac-7891-4bcc-a920-dde4077a...	local...	/	2027...	52						Medi...
Idea-70d5334f	5670eec4-b0f5-470f-9936-c1020313...	local...	/	2027...	49	✓		Strict			Medi...
JSESSIONID	D7FE6AB91EC8237A970DF2AF2B98A...	local...	/	Sessi...	42	✓					Medi...
REMEMBER_ME	2.1784812300.55056c3d691ae65cbe...	local...	/	2026...	88	✓					Medi...
XSRF-TOKEN	7a5fbf76-fe01-4a38-8994-a1c857860...	local...	/	Sessi...	46						Medi...

1. `/password/forgot` — 이메일 입력 화면



2. `/password/forgot` — 미가입 이메일 입력 시 안내



3. `/password/reset?token=...` — 무효/만료 토큰으로 접근 시 재요청 유도 화면

